

Informationen und Daten

Informatik · Klasse 9 · Material

Inhaltsverzeichnis

Material	3
M01 - Datensatzkarten	4
Aufgabe	4
Schüler A	4
Schüler B	4
Schüler C	4
Leitfragen	4
M02 - Klassen, Objekte und Attribute	5
Karten sortieren	5
Lösungsschema	5
Transfer	5
M03 - Primärschlüssel prüfen	6
Tabelle Schüler	6
Kriterien	6
Zusatzfrage	6
M04 - Fremdschlüssel und Beziehungen	7
Ausgangstabellen	7
Aufgaben	7
Merksatz	7
M05 - CRUD-Karten	8
Karte A	8
Karte B	8
Karte C	8
Karte D	8
Zuordnung	8
Wichtig	8
M06 - SQL-Befehlskarten	9
SELECT	9
WHERE	9
ORDER BY	9
INSERT	9
UPDATE	9
DELETE	9
CREATE TABLE	10
Aufgabe	10
M07 - SQL-Fehlersuche	11
A	11
B	11
C	11
D	11
E	11
M08 - Normalisierung	12
Ausgangstabelle	12
Aufgaben	12
Ziel	12
M09 - Denormalisierung	13

Situation	13
Auftrag	13
Merksatz	13
M10 - SQLite-Arbeitskarte	14
Ziel	14
1. Datenbank öffnen	14
2. Übersicht einschalten	14
3. Fremdschlüssel aktivieren	14
4. Tabellen anzeigen	14
5. Daten lesen	14
6. CRUD durchführen	14
7. Beenden	14
Hilfe	14
M11 - DB Browser for SQLite - Arbeitskarte	15
Ablauf	15
Übung	15
Wichtig	15
M12 - Datenqualität prüfen	16
Checkliste	16
Bewertung	16
M13 - Datenschutz-Fallkarten	17
Fall A	17
Fall B	17
Fall C	17
Fall D	17
M14 - SQL-Referenzkarte	18
Tabelle	18
Create	18
Read	18
Update	18
Delete	18
Sortieren	18
Eindeutigkeit	18
Beziehung	18
M15 - Peer-Review Datenmodell	19
Bewertetes Modell	19
Prüft	19
Eine Stärke	19
Eine Unklarheit	19
Ein Verbesserungsvorschlag	19

Material

Dieser Ordner enthält ergänzende Unterrichtsmaterialien zum Lernbereich **Informationen und Daten**.

Die Materialien unterstützen insbesondere:

- Datenmodellierung,
- Tabellen und Datensätze,
- Primär- und Fremdschlüssel,
- SQL und CRUD,
- Suchen, Filtern und Sortieren,
- Datenqualität,
- Datenschutz,
- Big Data und künstliche Intelligenz.

Für grundlegende SQL-Erklärungen wird auf [Grundlagen/SQL/](#) verwiesen.

Für die konkrete Arbeit mit SQLite wird auf [Werkzeuge/SQLite/](#) verwiesen.

M01 - Datensatzkarten

Aufgabe

Ordnet die Karten so, dass aus einzelnen Informationen sinnvolle Datensätze entstehen.

Schüler A

SchuelerID: 1001
Name: Mia
Klasse: 9a
Geburtsjahr: 2011

Schüler B

SchuelerID: 1002
Name: Tim
Klasse: 9a
Geburtsjahr: 2010

Schüler C

SchuelerID: 1003
Name: Lea
Klasse: 9b
Geburtsjahr: 2011

Leitfragen

1. Welche Attribute besitzt jeder Datensatz?
2. Welches Attribut eignet sich als Primärschlüssel?
3. Warum eignet sich der Name nicht zuverlässig als Primärschlüssel?
4. Welche weiteren Attribute könnten sinnvoll sein?

M02 - Klassen, Objekte und Attribute

Karten sortieren

Ordnet die Begriffe:

Schueler

Mia

Tim

SchuelerID

Name

Geburtsdatum

Klasse

9a

in die Kategorien:

- Klasse
- Objekt
- Attribut
- Attributwert

Lösungsschema

Begriff	Kategorie
---------	-----------

Transfer

Erstellt dieselbe Zuordnung für:

Buch

Kurs

Lehrkraft

Raum

M03 - Primärschlüssel prüfen

Entscheidet für jedes Attribut:

- geeignet,
- bedingt geeignet,
- ungeeignet.

Tabelle Schüler

Name

Geburtsdatum

E-Mail

Schuelernummer

SchuelerID

Kriterien

Ein guter Primärschlüssel sollte:

- eindeutig,
- stabil,
- möglichst kurz,
- immer vorhanden

sein.

Zusatzfrage

Welche Attribute könnten zusätzlich mit UNIQUE abgesichert werden?

M04 - Fremdschlüssel und Beziehungen

Ausgangstabellen

Klasse

KlasseID	Bezeichnung
1	9a
2	9b

Schüler

SchuelerID	Name	KlasseID
1001	Mia	1
1002	Tim	1
1003	Lea	2

Aufgaben

1. Markiert die Primärschlüssel.
2. Markiert den Fremdschlüssel.
3. Beschreibt die Beziehung.
4. Was wäre an folgendem Datensatz problematisch?

SchuelerID: 1004

Name: Noah

KlasseID: 99

Merksatz

Ein Fremdschlüssel verweist auf einen vorhandenen Datensatz einer anderen Tabelle.

M05 - CRUD-Karten

Schneidet die Karten aus und ordnet sie den vier CRUD-Operationen zu.

Karte A

```
INSERT INTO Schueler (SchuelerID, Name)
VALUES (1001, 'Mia');
```

Karte B

```
SELECT *
FROM Schueler
WHERE SchuelerID = 1001;
```

Karte C

```
UPDATE Schueler
SET Name = 'Mia Müller'
WHERE SchuelerID = 1001;
```

Karte D

```
DELETE FROM Schueler
WHERE SchuelerID = 1001;
```

Zuordnung

CRUD	SQL
Create	
Read	
Update	
Delete	

Wichtig

CREATE TABLE gehört nicht zum CRUD-Create. CRUD-Create meint das Anlegen eines **Datensatzes**.

M06 - SQL-Befehlskarten

SELECT

Daten lesen.

```
SELECT Name  
FROM Schueler;
```

WHERE

Daten filtern.

```
WHERE KlasseID = 1
```

ORDER BY

Daten sortieren.

```
ORDER BY Name ASC
```

INSERT

Datensatz anlegen.

```
INSERT INTO ...  
VALUES (...);
```

UPDATE

Datensatz ändern.

```
UPDATE ...  
SET ...  
WHERE ...;
```

DELETE

Datensatz löschen.

```
DELETE FROM ...  
WHERE ...;
```

CREATE TABLE

Tabelle anlegen.

CREATE TABLE ...

Aufgabe

Erstellt aus mehreren Karten eine sinnvolle SQL-Abfrage.

M07 - SQL-Fehlersuche

Findet die Fehler.

A

```
SELECT Name  
Schueler;
```

B

```
UPDATE Schueler  
Name = 'Mia'  
WHERE SchuelerID = 1;
```

C

```
DELETE Schueler  
WHERE SchuelerID = 1;
```

D

```
SELECT *  
FROM Schueler  
WHERE Name = Mia;
```

E

```
UPDATE Schueler  
SET KlasseID = 2;
```

Zusatzfrage

Die letzte Anweisung ist syntaktisch korrekt. Warum kann sie trotzdem gefährlich sein?

M08 - Normalisierung

Ausgangstabelle

SchuelerID	Name	Klasse	Klassenlehrer
1001	Mia	9a	Frau Müller
1002	Tim	9a	Frau Müller
1003	Lea	9b	Herr Weber

Aufgaben

1. Welche Daten werden mehrfach gespeichert?
2. Welche Probleme entstehen bei Änderungen?
3. Zerlegt die Tabelle in:

Schueler

Klasse

4. Ergänzt Primär- und Fremdschlüssel.

Ziel

Eine mögliche Struktur:

Klasse

- KlasseID
- Bezeichnung
- Klassenlehrer

Schueler

- SchuelerID
- Name
- KlasseID

M09 - Denormalisierung

Situation

Eine große Auswertung wird tausendfach pro Minute gelesen.

Für jede Anzeige sind mehrere Joins nötig.

Das Entwicklungsteam schlägt vor, einige bereits berechnete Werte zusätzlich zu speichern.

Auftrag

Bewertet den Vorschlag.

Vorteile

Nachteile

Risiken

Wann wäre die Entscheidung sinnvoll?

Merksatz

Redundanz kann bewusst eingesetzt werden, wenn der Nutzen die zusätzlichen Risiken und Pflegekosten rechtfertigt.

M10 - SQLite-Arbeitskarte

Ziel

Eine Datenbank öffnen, SQL ausführen und CRUD anwenden.

1. Datenbank öffnen

```
sqlite3 schule.db
```

2. Übersicht einschalten

```
.headers on  
.mode column
```

3. Fremdschlüssel aktivieren

```
PRAGMA foreign_keys = ON;
```

4. Tabellen anzeigen

```
.tables
```

5. Daten lesen

```
SELECT *  
FROM Schueler;
```

6. CRUD durchführen

- INSERT
- SELECT
- UPDATE
- DELETE

7. Beenden

```
.quit
```

Hilfe

Ausführliche Anleitung:

Werkzeuge/SQLite/

M11 - DB Browser for SQLite - Arbeitskarte

Ablauf

1. Datenbank öffnen oder neu anlegen.
2. Register **SQL ausführen** öffnen.
3. SQL-Anweisung eingeben.
4. Anweisung ausführen.
5. Ergebnis kontrollieren.
6. Änderungen speichern.

Übung

Führt aus:

```
SELECT *  
FROM Schueler;
```

Danach:

```
INSERT INTO Schueler  
    (SchuelerID, Name)  
VALUES  
    (2001, 'Alex');
```

Prüft anschließend erneut:

```
SELECT *  
FROM Schueler;
```

Wichtig

Die grafische Oberfläche unterstützt euch beim Arbeiten.

SQL soll trotzdem bewusst geschrieben und verstanden werden.

M12 - Datenqualität prüfen

Checkliste

Prüft einen Datensatz.

- ☐ Pflichtfelder vorhanden
- ☐ Datentypen plausibel
- ☐ eindeutige IDs
- ☐ keine unerwünschten Duplikate
- ☐ Fremdschlüssel gültig
- ☐ Werte im erlaubten Bereich
- ☐ Schreibweisen konsistent
- ☐ fehlende Werte bewusst behandelt

Bewertung

Gefundene Probleme

Folgen

Verbesserung

M13 - Datenschutz-Fallkarten

Fall A

Eine App speichert Namen und Geburtsdaten, obwohl für die Aufgabe nur eine zufällige Teilnehmer-ID benötigt wird.

Frage: Welche Daten könnten vermieden werden?

Fall B

Eine Teilnehmerliste liegt ungeschützt in einem öffentlich erreichbaren Ordner.

Frage: Ist dies eher ein Datenschutz-, Datensicherheits- oder beides Problem?

Fall C

Eine Statistik nutzt anonymisierte Daten.

Frage: Warum kann das datenschutzfreundlicher sein?

Fall D

Ein System speichert Daten unbegrenzt, obwohl sie nach Ende des Projekts nicht mehr benötigt werden.

Frage: Welche Alternative wäre sinnvoll?

M14 - SQL-Referenzkarte

Tabelle

```
CREATE TABLE Tabelle (  
    ID INTEGER PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL  
);
```

Create

```
INSERT INTO Tabelle (ID, Name)  
VALUES (1, 'Beispiel');
```

Read

```
SELECT *  
FROM Tabelle  
WHERE ID = 1;
```

Update

```
UPDATE Tabelle  
SET Name = 'Neu'  
WHERE ID = 1;
```

Delete

```
DELETE FROM Tabelle  
WHERE ID = 1;
```

Sortieren

```
ORDER BY Name ASC
```

Eindeutigkeit

```
UNIQUE
```

Beziehung

```
FOREIGN KEY (...)  
REFERENCES ...
```

Ausführliche Erklärung:

Grundlagen/SQL/

M15 - Peer-Review Datenmodell

Bewertetes Modell

Prüft

- ☐ Klassen/Tabellen sind sinnvoll gewählt.
- ☐ Attribute gehören zur richtigen Tabelle.
- ☐ Primärschlüssel sind eindeutig.
- ☐ Fremdschlüssel sind nachvollziehbar.
- ☐ unnötige Wiederholungen wurden vermieden.
- ☐ Beziehungen sind fachlich korrekt.

Eine Stärke

Eine Unklarheit

Ein Verbesserungsvorschlag
